

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №4
з курсу «Системи штучного інтелекту»
на тему:
«Двонаправлений пошук на графі»

Виконав:
студент групи ФЕІ-42
Трофимов Володимир
Перевірив:
ас. Іжик О. Б.

Зміст

Тема	3
Мета	3
Вхідні дані	3
Завдання	3
Критерії виконання роботи	3
Теоретичні відомості	5
1. Одиничний граф та його представлення у вигляді лабіринту.....	5
2. Двонаправлений хвильовий пошук.....	5
3. Організація «хвилі» у двонаправленому пошуку.....	5
4. Оператори переходу	7
5. Відмінності в організації циклу пошуку в порівнянні з однонаправленим.....	7
6. Умови завершення та відновлення знайденого шляху	8
7. Оцінювання результатів та теоретичні переваги двонаправленого пошуку	8
8. Блок-схема одно-направленого хвильового пошуку.....	9
Інтерфейс та принцип роботи програми	10
Результати дослідження роботи програми	13
<i>Дослідження 1: Дослідження двонаправленого пошуку з оператором «вверх-вниз-вправо-вліво»...</i>	13
<i>Дослідження 2: Дослідження двонаправленого пошуку з оператором переходу по діагоналях.</i>	14
<i>Дослідження 3: Дослідження пошуку з комбінованим оператором.</i>	15
<i>Дослідження 4: Порівняння операторів переходу та оцінка переваг двонаправленого пошуку перед однонаправленим</i>	16
<i>Дослідження 5: Дослідження впливу розміру графа на двонаправлений пошук.....</i>	17
<i>Дослідження 6: Дослідження впливу способу задання лабіринту (власний експеримент)</i>	18
Висновок	19
Код програми	20

Тема

Двонаправлений пошук на графі. Реалізація двонаправленого пошуку у хвильовому алгоритмі для знаходження найкоротшого шляху у одиничних графах.

Мета

Створити програму виконання двонаправленого хвильового пошуку на одиничному графі та дослідити особливості її роботи на графах різних розмірів та при різних умовах пошуку.

Вхідні дані

- одиничний граф порядку 10-20, представлений у вигляді лабіринту, який відображає матрицю суміжності графа;
- початкова і цільова вершини графу;
- оператор переходу.

Завдання

Створити програму виконання двонаправленого хвильового пошуку на одиничному графі, використовуючи одну з мов програмування високого рівня, в якій реалізувати:

- візуальне представлення створеного графу (у вигляді лабіринту, який представляє матрицю суміжності графа);
- знаходження найкоротшого шляху від вказаної початкової до цільової вершини, використовуючи алгоритм двонаправленого хвильового пошуку;
- виведення результатів пошуку.

Критерії виконання роботи

1. Створити програму виконання двонаправленого пошуку на основі хвильового алгоритму з використанням різних операторів переходу:

- «вверх-вниз-вправо-вліво»;
- «перехід по діагоналях»;
- комбінація двох вищеназваних операторів.

У програмі передбачити:

- створення графічного інтерфейсу, на якому відображається заданий одиничний граф (у вигляді лабіринту, де непрохідна ділянка лабіринту позначена -1, а прохідна – 0);
- можливість змінювати розмірність і порядок графу (відповідної матриці лабіринту та її прохідної і непрохідної частин);
- зміну початкової та цільової вершин графу;

- реалізувати візуалізацію процесу пошуку (поширення хвилі) і представлення його результатів; для кращої візуалізації процесу пошуку передбачити можливість зміни часу затримки;
- виведення результатів пошуку (вершин у знайденому шляху, кількості циклів розкриття вершин при його визначенні та кількості всіх розкритих вершин, а також часу, витраченого комп'ютером на здійснення пошуку) в окремому вікні (вікнах).

2. Дослідити особливості роботи пошуку за використання різних операторів переходу як при однакових решті умов пошуку – виду графу (лабіринту), однакових початкової та кінцевої вершин, так і зміні його розміру. Для цього:

- за допомогою створеної програми знайти найкоротший шлях з початкової до цільової точки створеного графу (лабіринту), використовуючи заданий оператор переходу;
- дослідити вплив на результат пошуку та його ефективність використання різних операторів переходу;
- відмітити переваги і недоліки використання кожного з операторів переходу;
- дослідити вплив на результат пошуку зміни розміру графу (співвідношення кількості прохідних і непрохідних клітинок лабіринту), на якому проводиться пошук;
- відмітити переваги використання двонаправленого пошуку в порівнянні з однонаправленим; пояснити, як і за рахунок чого вони отримуються.

3. Пояснити особливості модифікації пошуку в ширину при одно-направленому хвильовому пошуку, зокрема:

- як організовується «хвиля» у двонапрвленому пошуці?
- які відмінності у організації циклу мають місце для двонаправленого хвильового пошуку в порівнянні з одно-направленим?

4. Намалювати блок-схему організації двонаправленого хвильового пошуку.

Теоретичні відомості

1. Одиначний граф та його представлення у вигляді лабіринту

У даній лабораторній роботі розглядається одиначний граф, у якому кожному переходу між суміжними вершинами відповідає однакова вартість — одна одиниця. Тому довжина шляху визначається кількістю переходів між вершинами. Основною задачею пошуку є знаходження шляху від заданої початкової вершини до цільової з мінімальною кількістю переходів.

Для зручного візуального представлення одиначний граф задається у вигляді лабіринту — матриці клітинок. Прохідна клітинка позначається 0, а непрохідна — -1. Кожна прохідна клітинка відповідає вершині графа, а можливість переходу між двома сусідніми прохідними клітинками визначається вибраним оператором переходу.

Таким чином, матриця лабіринту фактично задає структуру графа: положення прохідних клітинок визначає множину вершин, а правила переходу визначають ребра між ними. Порядок графа в такому представленні відповідає кількості прохідних клітинок, тобто кількості вершин графа.

2. Двонаправлений хвильовий пошук

Двонаправлений хвильовий пошук — це модифікація хвильового алгоритму, яка полягає в одночасному зустрічному розгортанні двох хвиль пошуку:

- Прямої хвилі — від початкової вершини (Start) у напрямку до цільової;
- Зворотної хвилі — від цільової вершини (Target) назустріч початковій.

Поширення кожної хвилі відбувається рівнями (фронтами): спочатку розкриваються початкова та цільова вершини, потім усі доступні вершини на відстані одного переходу від них, далі — двох переходів і так далі. Кожна хвиля нарощує радіус огляду, доки вони не перетнуться у спільній вершині (точці зустрічі).

Замість того, щоб одна хвиля досліджувала граф на повну глибину відстані d , кожна з хвиль у двонаправленому пошуці проходить відстань близько $d/2$. Це дозволяє суттєво скоротити область переглянутих вершин лабіринту.

3. Організація «хвиль» у двонаправленому пошуку

Основним принципом алгоритму є паралельна або почергова обробка вершин двох зустрічних фронтів. Для цього використовується пара черг типу FIFO (First In — First Out):

- Queue_Start — черга для вершин прямої хвилі;
- Queue_Target — черга для вершин зворотної хвилі.

Для кожної хвилі підтримується власна структура стану:

- масив або словник номерів хвиль (відстаней): Distance_Start та Distance_Target;
- структури для збереження батьківських вершин: Parent_Start (показує, звідки прийшла пряма хвиля) та Parent_Target (показує, звідки прийшла зворотна хвиля).

Початкова вершина поміщається до Queue_Start з відстанню 0, а цільова — до Queue_Target з відстанню 0. Під час роботи алгоритму на кожному кроці з однієї або обох черг вибираються вершини, і для них генеруються дозволених сусіди. Якщо сусідня клітинка є прохідною і ще не відвідана поточною хвилею, їй призначається номер хвилі на одиницю більший за поточний, фіксується батьківська вершина і вона додається у відповідну чергу.

Схематично поширення зустрічних хвиль та їх зустріч можна представити як на рисунку 1.

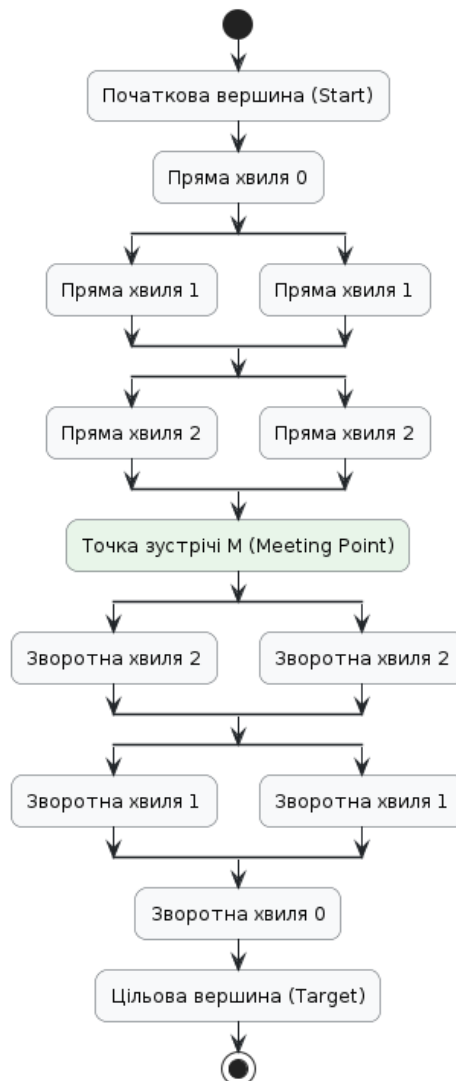


Рис.1 – Схеми роботи двонаправленого хвильового пошуку (зустріч прямої та зворотної хвиль).

На практиці хвиля може поширюватися в різних напрямках залежно від структури лабіринту та вибраного оператора переходу.

4. Оператори переходу

У лабораторній роботі досліджуються три варіанти оператора переходу.

Оператор «вверх-вниз-вправо-вліво» дозволяє переходити лише до чотирьох безпосередньо сусідніх клітинок: вгору, вниз, вправо та вліво. Діагональні переходи при цьому заборонені. Такий оператор відповідає класичному переміщенню по клітинках прямокутної сітки.

Оператор переходу по діагоналях дозволяє переміщення лише до чотирьох діагонально сусідніх клітинок: вгору-вліво, вгору-вправо, вниз-вліво та вниз-вправо. Горизонтальні та вертикальні переходи в цьому режимі не використовуються.

Комбінований оператор об'єднує попередні два варіанти. Дозволяється перехід у всіх восьми напрямках: чотирьох вертикально-горизонтальних і чотирьох діагональних.

Зміна оператора переходу змінює структуру зв'язків між вершинами графа. Тому навіть для одного й того самого лабіринту зміна оператора може призвести до іншої довжини знайденого шляху, іншої кількості відкритих вершин та іншої кількості хвиль.

5. Відмінності в організації циклу пошуку в порівнянні з однонаправленим

Організація ітераційного циклу у двонаправленому хвильовому пошуці має суттєві відмінності від однонаправленого пошуку:

- Умова продовження циклу: у той час як в однонаправленому алгоритмі цикл продовжується до спустошення однієї черги, у двонаправленому циклі пошук ведеться, доки обидві черги не є порожніми (Queue_Start не порожня ТА Queue_Target не порожня). Якщо хоча б одна з черг вичерпалася без перетину хвиль, це гарантує відсутність зв'язку між стартом і ціллю.
- Стратегія розкриття вершин: на кожній ітерації головного циклу обробляється крок або шар прямої хвилі, а потім — крок або шар зворотної хвилі (або обирається черга з меншим числом вершин для балансування).
- Критерій перевірки завершення (зустріч хвиль): в однонаправленому пошуці зупинка відбувається тоді, коли поточна вершина збігається з цільовою ($v == Target$). У двонаправленому пошуці завершення фіксується тоді, коли чергова вершина, що розкривається однією хвилею, вже була відвідана зустрічною хвилею (тобто міститься у множині Visited протилежного напрямку). Ця спільна вершина M стає точкою зустрічі.

Схематичне порівняння циклів роботи однонаправленого та двонаправленого підходів наведено на рисунку 2.

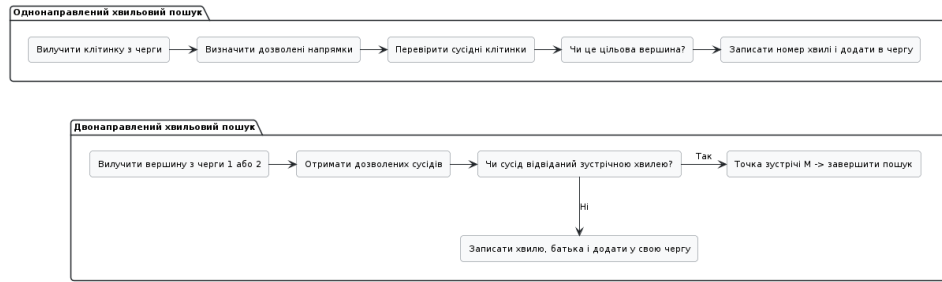


Рис.2 – Цикл роботи однонаправленого та двонаправленого хвильового пошуку.

6. Умови завершення та відновлення знайденого шляху

Пошук завершується в одному з двох випадків:

- Успішне знаходження шляху: зафіксовано спільну вершину M , до якої дійшли обидва фронти. Після цього алгоритм зупиняє розгортання хвиль і формує повний маршрут шляхом «зшивання» двох ділянок:
 - Перша половина шляху розгортається від точки зустрічі M у зворотному напрямку до початкової вершини S за збереженими зв'язками Parent_Start .
 - Друга половина шляху розгортається від точки зустрічі M до цільової вершини T за зв'язками Parent_Target . Остаточний маршрут є конкатенацією цих двох відрізків. Його загальна довжина визначається сумою: $L = \text{Distance_Start}[M] + \text{Distance_Target}[M]$.
- Шлях не існує: хоча б одна з черг стала порожньою раніше, ніж хвилі зустрілися. Це означає, що початкова і цільова вершини знаходяться в різних незв'язних компонентах графа-лабіринту.

7. Оцінювання результатів та теоретичні переваги двонаправленого пошуку

Для оцінки ефективності алгоритму використовуються такі параметри:

- Довжина шляху (кількість вершин / переходів у результуючому маршруті);
- Кількість розкритих вершин (сумарна кількість клітинок, опрацьованих прямою та зворотною хвилями);
- Кількість хвильових циклів (кількість пройдених рівнів до зустрічі);
- Час роботи алгоритму (в мілісекундах).

Головна перевага двонаправленого пошуку над однонаправленим полягає у суттєвому скороченні обчислювальної складності: якщо середній коефіцієнт розгалуження графа становить b , а відстань між стартом і метою дорівнює d :

- Однонаправлений пошук у ширину в гіршому випадку генерує та досліджує кількість вершин порядку $O(b^d)$.
- Двонаправлений пошук розгортає два кола/фронти радіусом приблизно $d/2$, внаслідок чого обсяг дослідженого простору станів оцінюється як $O(b^{d/2} + b^{d/2}) = O(2 \cdot b^{d/2})$.

Завдяки цьому двонаправлений хвильовий пошук вимагає значно менше циклів обробки і пам'яті, що дає відчутний виграш у часі на матрицях великого розміру.

8. Блок-схема одно-направленого хвильового пошуку

Блок-схема організації алгоритму, зображена на рисунку 3, відображає ініціалізацію двох черг (Queue_Start та Queue_Target), почергове або збалансоване вилучення вершин, перевірку перетину з протилежною хвилею, визначення точки зустрічі та зшивання двох гілок маршруту.

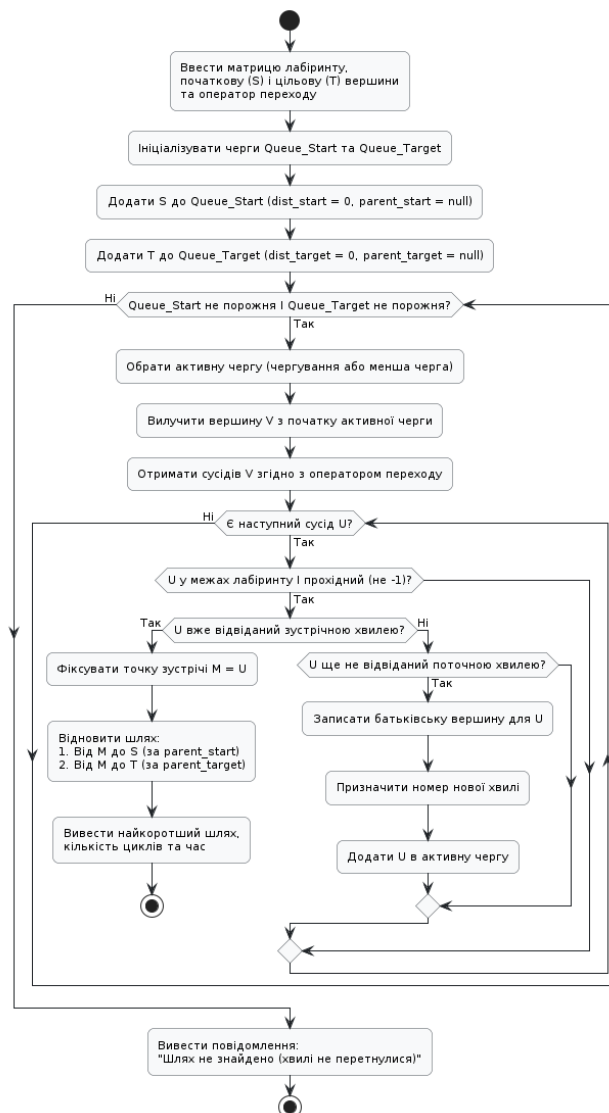


Рис.3 – Блок-схема організації алгоритму двонаправленого хвильового пошуку

Інтерфейс та принцип роботи програми

Графічний інтерфейс розробленої програми реалізовано мовою Python з використанням бібліотеки CustomTkinter, яка забезпечує створення сучасних елементів графічного інтерфейсу та їх взаємодію з алгоритмічною частиною програми. Структуру інтерфейсу організовано у вигляді трьох основних функціональних областей: панелі навігації між вкладками, панелі керування параметрами графа-лабіринту та алгоритму однонапрявленого пошуку і області візуалізації графа та процесу пошуку.

Основна функціональність програми зосереджена у вкладці «Лабораторна», яка призначена для виконання експериментів та дослідження особливостей алгоритму однонапрявленого хвильового пошуку. Додатково передбачено вкладки «Результати» та «Теорія» (див. рис.4).

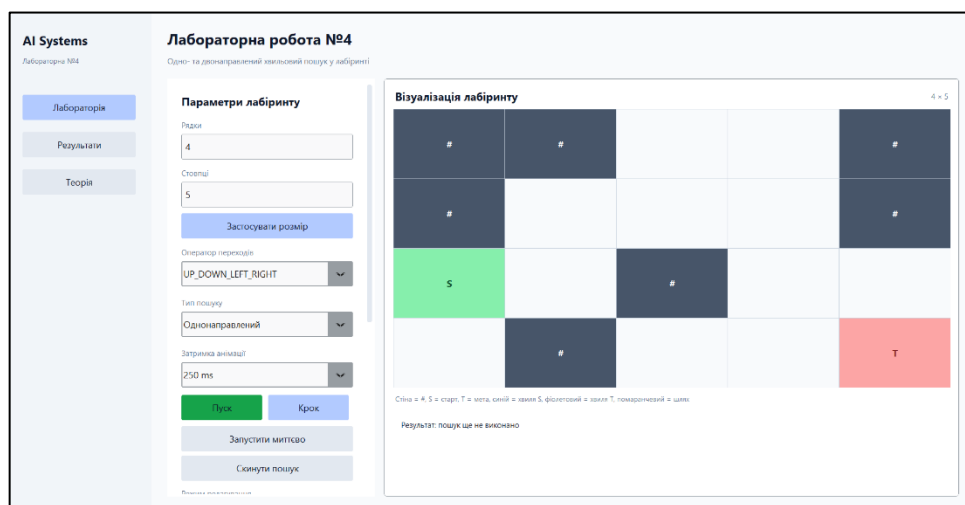


Рис.4 – Загальний вигляд інтерфейсу.

Вкладка «Лабораторна» є основним робочим середовищем програми та містить засоби налаштування параметрів експерименту, керування графом-лабіринтом, запуску алгоритму пошуку та його візуалізації.

У секції «Параметри лабіринту» користувач може вибрати тип обходу по лабіринту («вверх-вниз-вправо-вліво», «перехід по діагоналях», комбінація двох попередніх операторів), вибрати тип пошуку (одно чи двонаправлений) та задавати основні параметри лабіринту (кількість клітинок у висоту та ширину).

Також задаються початкова (Start) та цільова (Target) вершини пошуку. Їх можна задати натисненням на графі правою кнопкою миші на клітинку (початок), а натисненням правою кнопкою миші та ctrl – цільову клітинку. Для проведення експериментів зі зміною напрямку пошуку передбачено функцію «Поміняти S/T», яка автоматично змінює початкову та цільову вершини місцями.

Для керування швидкістю демонстрації алгоритму передбачено параметр часу виконання одного кроку. Він використовується під час покрокової анімації

пошуку та дозволяє спостерігати послідовність розкриття вершин і зміни стану графа.

Основні операції керування пошуком представлені такими функціями (див. рис.5):

- «Пуск» — виконує пошук алгоритмом однонаправленим хвильовим пошуком від заданої початкової вершини до цільової;
- «Крок» — виконує один етап алгоритму, що дає змогу детально простежити послідовність роботи алгоритму;
- «Запустити миттєво» — виконує пошук та відображає кінцевий результат без покрокової візуалізації;
- «Скинути пошук» — видаляє результати поточного пошуку та повертає лабіринт до початкового стану.

Рис.5 –Вигляд (основної частини) параметрів експерименту.

Для дослідження різних конфігурацій графа реалізовано секцію для редагування графа. За її допомогою можна змінювати структуру графа шляхом зміни початкових та цільової клітинок, створення та видалення перешкод. Якщо користувач хоче використовувати тільки ліву кнопку миші, то надається список із вибору поточної дії та поділа використовувати її графічно на лабіринті. Також можна використовувати такі комбінації:

- Ліва кнопка миші – виконувати поточну дію, зазначеної в списку;
- Ліва кнопка миші + shift – інвертувати поточну дію;
- Права кнопка миші – перемістити початкову клітинку;
- Права кнопка миші + ctrl – вибрати цільову клітинку.

Крім того, передбачено можливість генерування випадкового графа. Дана функція має допоміжний характер і використовується для додаткового

тестування працездатності програмної реалізації. Основний граф, використаний для виконання лабораторного дослідження, задається явно із заздалегідь визначеним лабіринтом, що відповідає вимогам лабораторної роботи. Також при генерації лабіринту можна вибрати спосіб заповнення: випадковий, місто, природа та каньйон.

Для швидкого повернення до початкової конфігурації передбачено функцію «Стандартний лабіринт». Вона відновлює вихідну структуру графа після виконання експериментів з зміною лабіринту.

Центральну частину робочого вікна займає область візуалізації графа-лабіринту. Лабіринт представлений у вигляді таблиці із клітинками, куди можна перейти чи ні, а також початкової клітинки (зелений колір) та кінця (червоний колір).

Під час виконання алгоритму візуально відображаються проміжні стани алгоритму (див. рис.6). Це дозволяє простежити порядок розкриття клітинок, формування черги та поступове наближення до цільової клітинки. Зміна візуального стану клітинки використовується для розмежування невідвіданих, відвіданих, поточної та розкритої вершин. Після успішного завершення пошуку знайдений маршрут додатково виділяється на графі.

Такий підхід забезпечує наочне представлення алгоритму та дозволяє безпосередньо співвіднести теоретичний принцип пошуку з фактичною послідовністю обробки вершин.

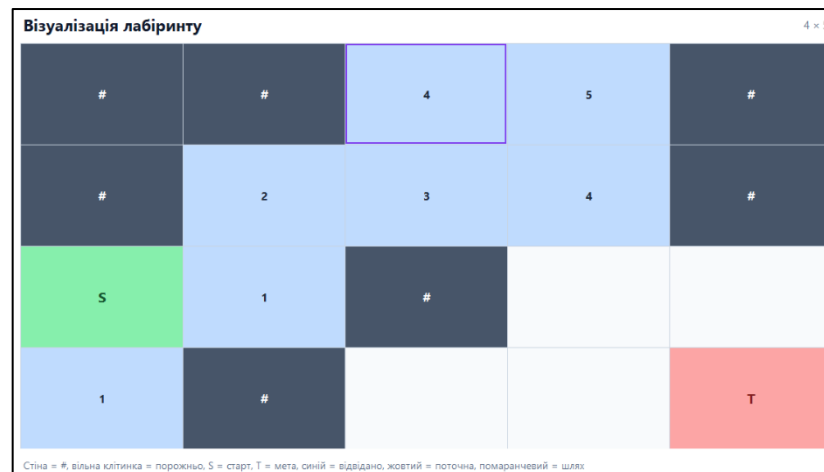


Рис.6 – Візуалізація графа

Вкладка «Результати» призначена для фіксації результатів проведених експериментів. Результати можуть зберігатися у вигляді окремих записів, що дозволяє порівнювати різні конфігурації графа-лабіринта та параметри пошуку.

Додавання запису до таблиці результатів здійснюється автоматично після кожного пуску алгоритму (див. рис.7).

Результати									
Таблиця експериментів									
Розмір	Кількість	Оператор	Старт	Мета	Довжина шляху	Хвилювальні цикли	Відбито	Час виконання	
4 x 5	13	UP_DOWN_LEFT_RIGHT	20	34	7	7	10	2314.457 ms	

Рис.7 – Вкладка «Результати»

Вкладка «Теорія» (див. рис.8) містить стислий довідковий матеріал щодо алгоритмів пошуку. У ній наведено основний принцип роботи, особливості використання, порядок розкриття вершин, умови знаходження найкоротшого шляху та основні характеристики алгоритму.

Наявність теоретичного розділу безпосередньо в програмному середовищі забезпечує можливість оперативного звернення до необхідних теоретичних відомостей під час проведення експериментів.

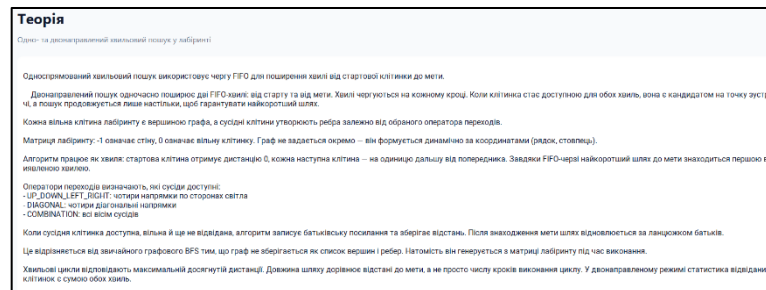


Рис.8 – Вкладка «Теорія»

Таким чином, розроблений графічний інтерфейс об'єднує засоби формування та редагування графа-лабіринту, налаштування параметрів пошуку, покрокового виконання, візуального спостереження за його роботою та фіксації отриманих результатів. Це забезпечує необхідну функціональність для проведення досліджень впливу розміщення перешкод на графі, його розміру, способу обходу суміжних вершин та напрямку пошуку на результати роботи алгоритмів пошуку.

Результати дослідження роботи програми

Дослідження 1: Дослідження двонаправленого пошуку з оператором «вверх-вниз-вправо-вліво».

У першому досліді було виконано пошук найкоротшого шляху на заданому лабіринті з використанням оператора переходу «вверх-вниз-вправо-вліво». Заданий лабіринт має розмір 10x15. Початкова (координати (0,0)) та цільова (координати (9,14)) клітинки, а також структура лабіринту залишалася незмінними протягом дослідів. При використанні цього оператора перехід

можливий лише між клітинками, які мають спільну сторону. Діагональні переходи не допускаються.

Після запуску пошуку дві хвилі починають поширюватися одночасно назустріч одна одній: пряма хвиля стартує з початкової вершини S , а зворотна хвиля — з цільової вершини T . Обидві черги обробляються синхронно крок за кроком, послідовно розкриваючи доступні клітинки та нарошуючи фронти з відповідними номерами хвиль. Пошук завершується в момент, коли одна з хвиль досягає клітинки, яка вже була відкрита зустрічною хвилею, фіксуючи точку зустрічі M . Після цього алгоритм відновлює шлях у двох напрямках: від точки M до S за батьківськими зв'язками прямої хвилі та від точки M до T за зв'язками зворотної хвилі.

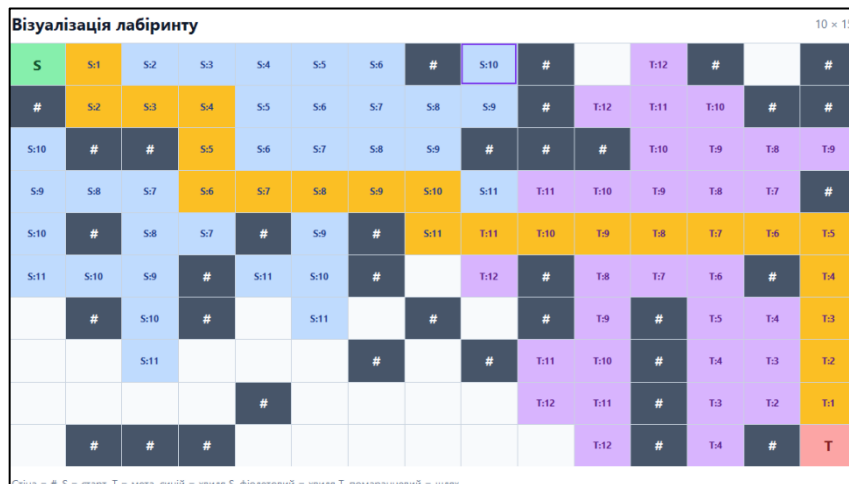


Рис.9 – Двонаправлений обхід лабіринту оператором «вверх-вниз-вправо-вліво» (зустріч прямої та зворотної хвиль)

За результатами дослідів (представлений на рисунку 9) було отримано найкоротший шлях між заданими вершинами. Кількість відкритих вершин рівна 88, а шлях складався із 23 вершин. Частина відкритих вершин не входила до остаточного шляху, що показує ефективність перед перебором всім вершин графа. Час виконання пошуку становив 0,391 мс.

Таким чином, оператор «вверх-вниз-вправо-вліво» забезпечує пошук лише через горизонтальні та вертикальні переходи. Він є зручним для представлення класичного переміщення по клітинковому лабіринту, однак у випадках, коли короткий маршрут проходить по діагоналі, такий маршрут цим оператором знайдений не буде.

Дослідження 2: Дослідження двонаправленого пошуку з оператором переходу по діагоналях.

У другому досліді для того самого лабіринту та тих самих початкової і цільової вершин було використано оператор переходу по діагоналях. У цьому

режимі дозволені переходи лише між діагонально сусідніми клітинками. Горизонтальні та вертикальні переходи не використовуються.

Поширення хвиль відбувалося аналогічно до першого досліду, однак множина доступних сусідів кожної вершини змінилася. Через це змінилася і структура графа, яка фактично формується поверх тієї самої матриці лабіринту.

Якщо між початковою та цільовою вершинами існує діагональний шлях, алгоритм знаходить найкоротший шлях саме серед дозволених діагональних переходів. Якщо ж необхідний для досягнення цілі перехід потребує горизонтального або вертикального переміщення, цільова вершина може залишитися недосяжною навіть у випадку, коли для першого оператора шлях існував.

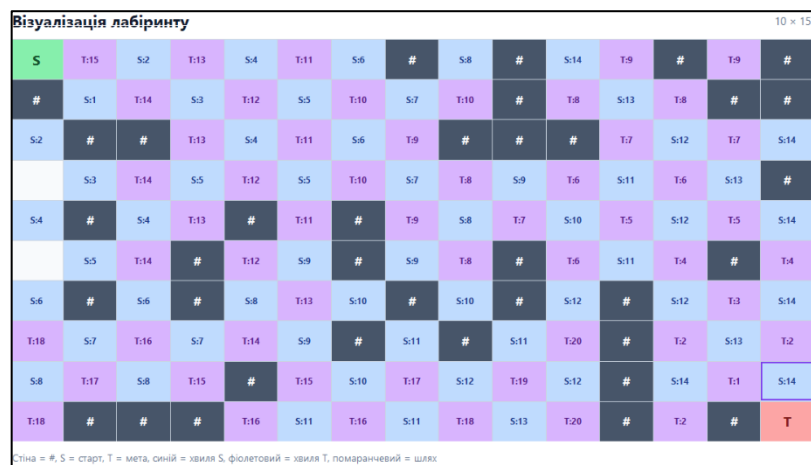


Рис.10 – Двонаправлений обхід лабіринту оператором переходу по діагоналях

Для даного лабіринту згідно рисунка 10 не було знайдено шляху оскільки початкова та цільова вершини не знаходилися на діагоналях спільних діагоналях.

Порівняння результатів із першим дослідом показало, що зміна оператора переходу без зміни самого лабіринту фактично змінює граф, на якому виконується пошук. Тому можуть змінюватися довжина шляху, кількість відкритих вершин та кількість циклів поширення хвилі.

Дослідження 3: Дослідження пошуку з комбінованим оператором.

У третьому досліді для того самого лабіринту та тих самих початкової і цільової вершин було використано комбінований оператор. Він дозволяє здійснювати переходи у восьми напрямках: вгору, вниз, вправо, вліво та по чотирьох діагоналях.

Завдяки збільшенню кількості можливих переходів хвиля отримує більше варіантів поширення. При цьому алгоритм все одно обробляє вершини за рівням, тому знайдений шлях є найкоротшим відносно саме цього оператора переходу.

Для одного й того самого лабіринту комбінований оператор може забезпечити шлях, який містить меншу кількість переходів, ніж шлях при використанні лише горизонтальних і вертикальних переміщень. Це пов'язано з тим, що один діагональний перехід може замінити послідовність горизонтального та вертикального переходів.

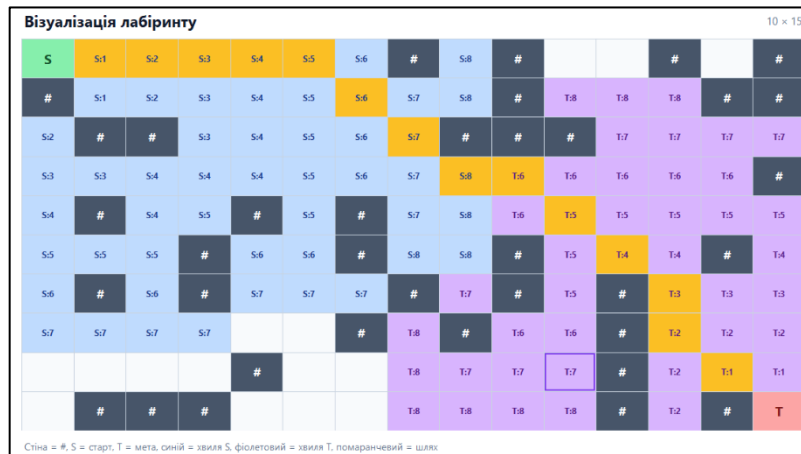


Рис.11 – Двонаправлений обхід лабіринту комбінованим оператором переходу

Результат, представлений на рисунку 11, показав зміну шляху відносно оператора в досліді 1, а також зменшилася довжина шляху: з 23 на 15. Час при цьому збільшився до 0.609 мс.

Дослідження 4: Порівняння операторів переходу та оцінка переваг двонаправленого пошуку перед однонаправленим

У четвертому досліді було проведено серію експериментів для трьох операторів переходу на фіксованому лабіринті при різних парах початкових і цільових вершин. Крім того, для демонстрації ефективності алгоритму за методичкою виконано пряме зіставлення характеристик однонаправленого (1-W) та двонаправленого (2-W) пошуку.

Для кожного варіанта були отримані такі характеристики:

Оператор переходу	Початкова вершина	Цільова вершина	Довжина шляху	Хвильові цикли (1-W / 2-W)	Відкриті вершини (1-W / 2-W)	Час пошуку, мс (1-W / 2-W)
Вверх-вниз-вправо-вліво	(0,0)	(4,9)	13	13/9	61/64	0.247/0.225
	(9,4)	(0,10)	17	17/10	110/75	0.454/0.375
	(9,14)	(0,4)	19	19/12	85/89	0.344/0.386
Діагональний	(0,0)	(4,9)	-	14/14	57/112	0.252/0.501
	(9,4)	(0,10)	-	16/16	55/112	0.250/0.522
	(9,14)	(0,4)	-	20	55/112	0.253/0.499
Комбінований	(0,0)	(4,9)	9	9/6	62/83	0.358/0.438
	(9,4)	(0,10)	10	10/6	101/74	0.626/0.429
	(9,14)	(0,4)	11	11/7	77/76	0.445/0.429

Результати показують, що оператор переходу безпосередньо впливає на структуру доступних шляхів. При чотирьох напрямках пошук обмежений горизонтальними та вертикальними переміщеннями. Діагональний оператор формує інший набір можливих маршрутів, однак не завжди знаходить шлях (а в моєму випадку для трьох випадків він не знайшов шлях), а комбінований оператор дозволяє використовувати обидва типи переходів, що зменшує шлях.

При цьому необхідно враховувати, що довжини шляхів, отримані для різних операторів, не завжди доцільно порівнювати як звичайну геометричну відстань, оскільки сам набір дозволених переходів є різним. У кожному випадку алгоритм знаходить найкоротший шлях відповідно до вибраної моделі переходів.

Щодо аналізу результатів між одно- та двонапрямленого пошуку, то не можна сказати точного висновку, однак в певних випадках двонапрямлений пошук був ефективнішим, а в інших однонапрямлений. Це зумовлено невеликою відстанню між початковою та цільовою вершинами.

Дослідження 5: Дослідження впливу розміру графа на двонаправлений пошук

Для дослідження впливу розміру графа було змінено розмір матриці лабіринту та відповідно кількість її прохідних клітинок. При цьому принцип побудови лабіринту та оператор переходу залишалися однаковими.

Для кожного розміру графа виконувалося окреме знаходження шляху. Початкова та цільова вершини знаходяться в лівому верхньому та правому нижньому куті відповідно. Вибрано комбінований спосіб обходу вершин. Результати доцільно представити у таблиці:

Розмір лабіринту	Кількість вершин	Довжина шляху	Цикли хвили	Відкриті вершини	Час пошуку (мс)
15 × 20	217	21	11	175	1.085
20 × 30	434	30	16	342	2.133
30 × 50	1076	48	25	891	5.837

При збільшенні розміру графа потенційно збільшується область, яку може досліджувати алгоритм. Тому за інших однакових умов може збільшуватися кількість відкритих вершин та час виконання пошуку. Однак конкретна кількість відкритих вершин залежить не тільки від розміру матриці, а й від розташування непрохідних клітинок, початкової та цільової вершин.

Тому збільшення розміру лабіринту не обов'язково призводить до пропорційного збільшення часу або кількості відкритих вершин.

Дослідження 6: Дослідження впливу способу задання лабіринту (власний експеримент)

Для дослідження впливу способу задання лабіринту було створено список із дозволених генерацій графа: випадково, місто, природа, каньйон. При цьому розмір графа, оператор переходу та початкова й цільова вершини залишалися однаковими.

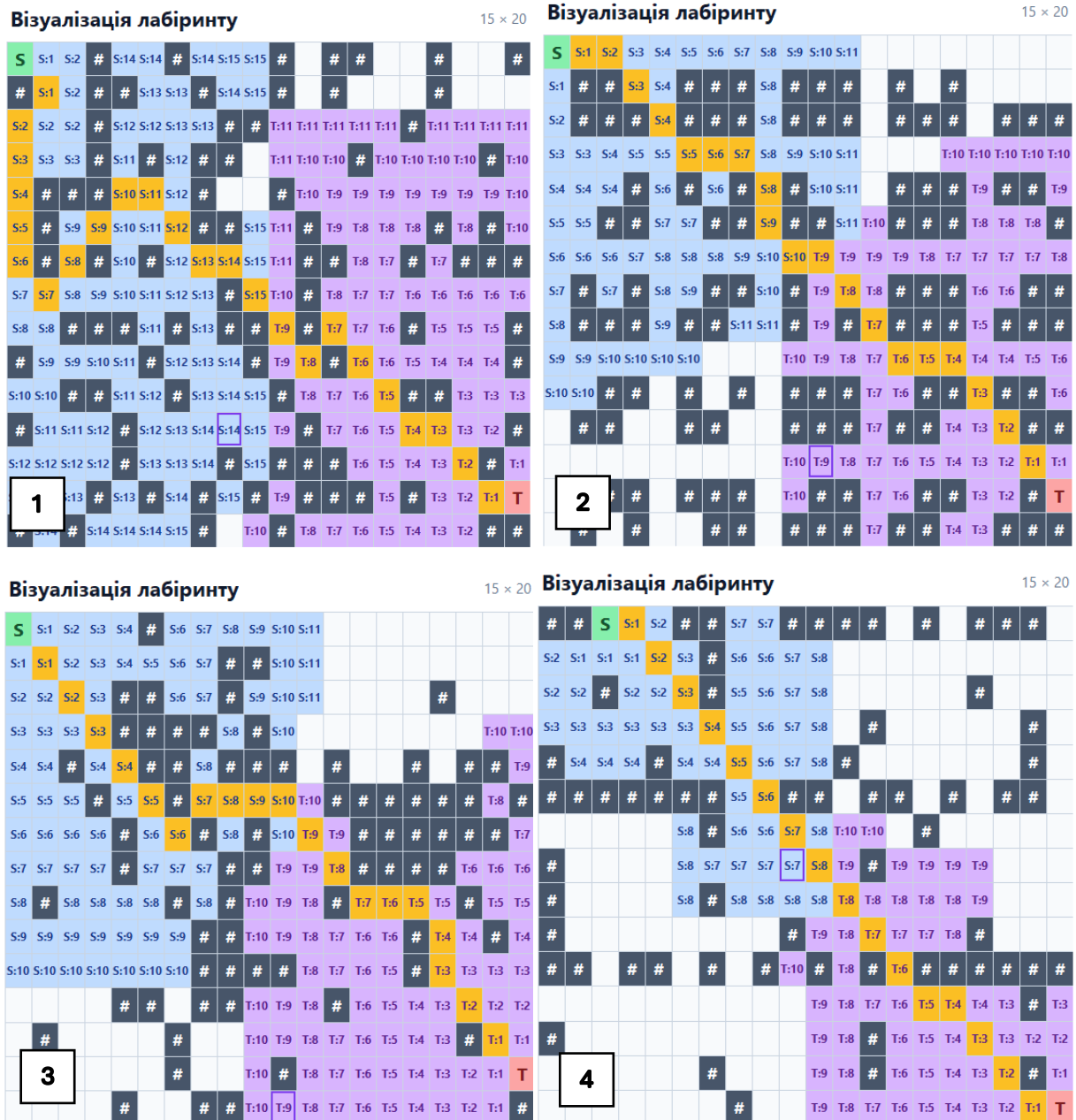


Рис.12 – Обхід лабіринту оператором комбінованого переходу при різних генерації графу: 1 – випадково, 2 – місто, 3 – природа, 4 - каньйон.

Результати показують, що спосіб завдання непрохідних клітинок безпосередньо впливає на структуру доступних шляхів. Найкоротші шляхи (див. рис. 12) є на лабіринті 4 (довжина шляху - 16), а найдовший на лабіринті 1 (довжина шляху 24).

Висновок

У ході лабораторної роботи було розроблено та досліджено програму двонаправленого хвильового пошуку найкоротшого шляху в одиничному графі, представленому у вигляді лабіринту. У програмі реалізовано візуальне представлення прохідних і непрохідних клітинок, вибір початкової та цільової вершин, поширення хвиль та відновлення знайденого шляху.

Було досліджено три оператори переходу: «вверх-вниз-вправо-вліво», діагональний та комбінований. Проведені експерименти показали, що вибір оператора безпосередньо впливає на структуру графа, можливість досягнення цільової вершини, довжину знайденого шляху, кількість відкритих вершин та час виконання пошуку. При використанні комбінованого оператора з'являється можливість здійснювати діагональні переходи, що в окремих випадках дозволяє скоротити кількість переходів до цільової вершини. Діагональний оператор, навпаки, обмежує пошук лише діагональними напрямками, тому для окремих пар вершин шлях може бути відсутнім.

Також було досліджено вплив розміру графа. Для використаних у роботі прикладів зі збільшенням розміру лабіринту збільшувалися кількість відкритих вершин і час виконання алгоритму. Дослідження різних способів генерації лабіринту показало, що розташування прохідних і непрохідних клітинок також істотно впливає на довжину маршруту та процес поширення хвиль.

Аналіз різниці між одно- та двонапрямленим пошуком показало, що перевагу в певних випадках має двонапрямлений пошук, хоча для кращих результатів досліджень необхідно провести їх на великому графі, щоб побачити суттєву перевагу двонапрямленого над одно-напрямленим пошуком (згідно з теорії).

Отже, проведені експерименти підтвердили, що двонаправлений хвильовий пошук забезпечує знаходження найкоротшого шляху в межах заданої моделі переходів. Використання різних операторів переходу та різних структур лабіринту дозволяє досліджувати вплив способу формування графа на ефективність і результат роботи алгоритму.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/AI_systemem у директорії lab_4 та має повну інструкцію запуску даної програми.